

Laborator 2.

Medii de dezvoltare software și limbaje de programare pentru aplicații de timp real.

IAR pentru AVR.

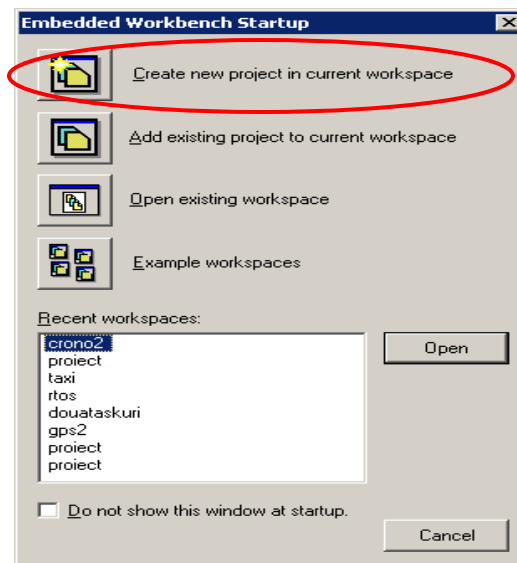
Medii de simulare/depanare aplicații de timp real.

(IAR Embedded Workbench pentru AVR)

1. Mediul de dezvoltare IAR Embedded WorkBench. Limbajul C pentru micro sisteme.

Este folosit pentru dezvoltarea aplicațiilor pentru microcontrolerele din familia ATMEGA printre care se află și ATMEGA128. Descrierea aplicațiilor se face folosind o versiune a limbajului C standard.

Descrierea unei aplicații se face în primul rând prin crearea unui spațiu de lucru (la versiunea IAR4.11 care se utilizează în laborator). Pentru crearea spațiului de lucru, la deschiderea aplicației se accesează opțiunea „Create new project in recent workspace” din fereastra de întâmpinare sau din meniul File – New – Workspace apoi Project – Create New Project.



Se dă OK la următoarea fereastră care apare (Create new project).

În continuare se salvează numele proiectului în directorul ales.

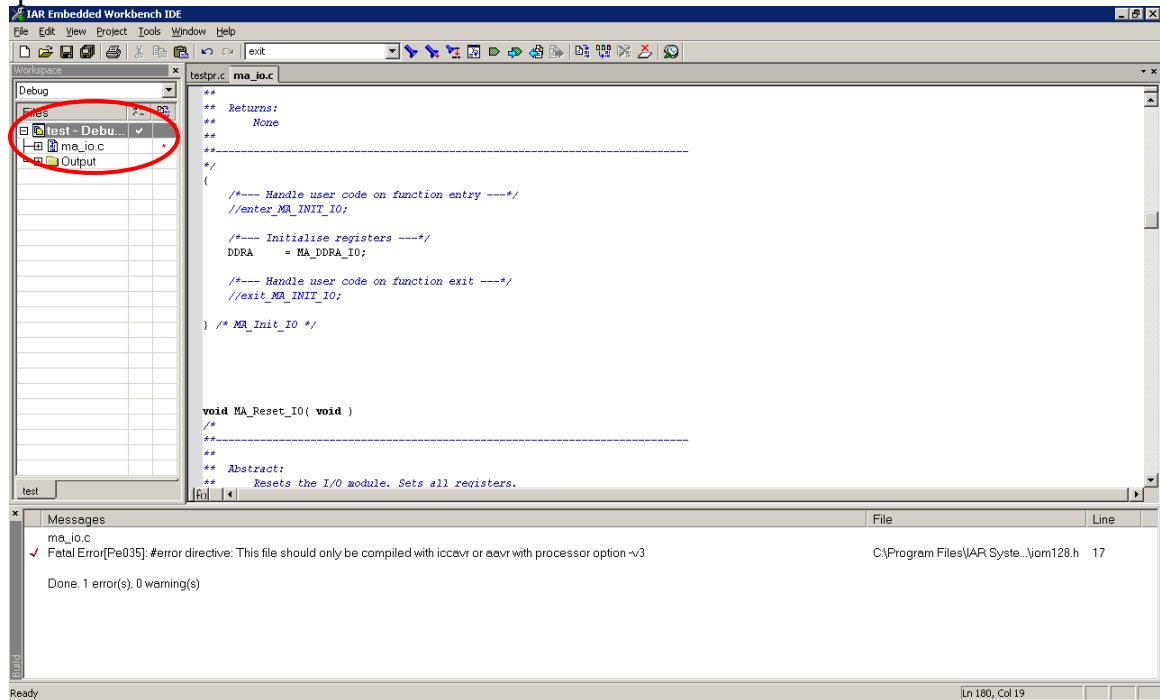
La proiectul nou creat se pot adăuga fișiere deja create (cu aplicația MakeAPP de exemplu) sau se pot crea fișiere noi.

Pentru adăugarea unui fișier deja creat (în cazul nostru cu extensia .c) se accesează din meniul Project – Add files și se selectează fișierele care se doresc adăugate.

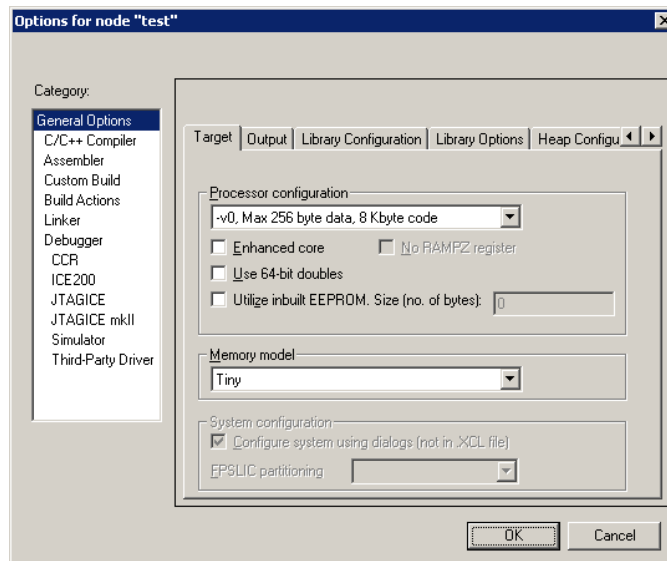
Pentru crearea unui fișier nou se selectează din meniul File – New – File . Noul fișier creat se salvează cu extensia dorită, în cazul nostru .c.

După ce se includ fișierele existente sau se va introduce codul sursă în fișierele noi se vor face setările proiectului. Astfel, din meniul se va selecta Project – Options având grija

ca în stânga, în fereastra Workspace să fie selectat proiectul și nu unul din fișierele componente.



La selectarea Options din Project va fi afișată următoarea fereastră:

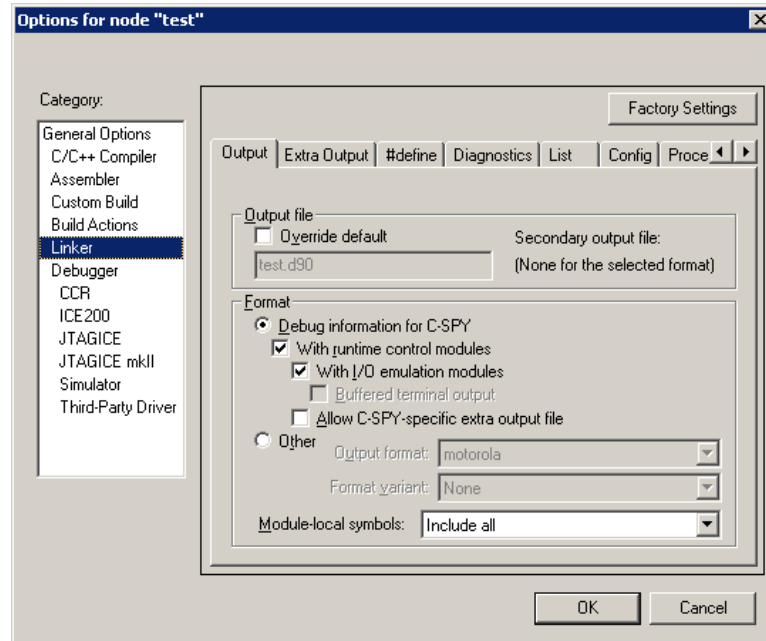


Aici se pot face toate configurațiile legate de proiect cum ar fi: tipul microcontrolerului folosit, configurația de memorie, tipul instrumentului de simulare/depanare folosit, tipul fișierului obiect.

Se poate observa că există mai multe categorii de opțiuni, afișate în partea stângă. Prima din acestea este General option. Aici se vor seta, în tag-ul Target în „Processor configuration” din lista derulată „cpu = m128 Atmega128” deci tipul care va fi utilizat.

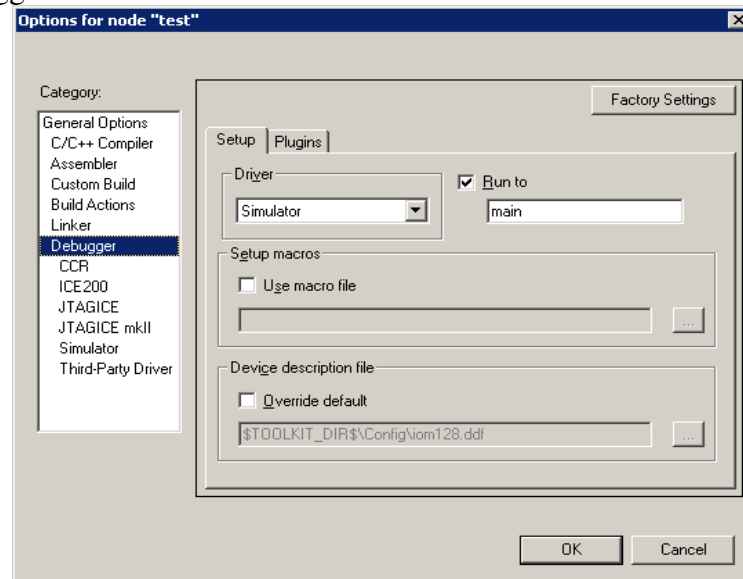
La „Memory model” se va selecta „Small” pentru a permite operarea cu memoria RAM internă (salvarea variabilelor declarate global se va face în memoria SRAM internă).

O altă setare importantă o reprezintă stabilirea tipului de fișier generat în urma compilării și realizării legăturilor între module. Aceasta se stabilește din categoria de opțiuni „Linker”.



La „Format” se poate configura ca formatul pentru simulare/depanare („Debug information for CSpy”) sau alt format folosit de alte aplicații („other”).

O altă setare este aceea a instrumentului folosit pentru testarea și programarea microcontorler-ului. Acesta poate fi un simulator soft sau un instrument hard de programare și testare cu monitorizarea „on-line” a datelor. Setarea se face accesînd categoria „Debugger”.



Din „Driver” se vor selecta tipul instrumentului folosit.

2. Limbajul C pentru microcontrolere din familia AVR

Pentru programele descrise în cele ce urmează se va folosi o versiune a limbajului C standard cu următoarele particularități:

a. Tipuri de variabile

Variabilele folosite pot fi de următoarele tipuri, fiind declarate folosind sintaxa clasică întâlnită în C sau într-o formă alternativă conform tabelului de mai jos:

C standard	Alternativ	Număr octeți	Domeniu
Char	S8	1	-127..127
Unsigned char	U8	1	0..255
Int	S16	2	-32767..32767
Unsigned int	U16	2	0..65535
Long	S32	4	-2147483647.. 2147483647
Unsigned long	U32	4	0..4294967295
Float	-	4	cu virgulă (10^{-5})
double	-	8	cu virgulă (10^{-5})

b. Stocare variabile

Există mai multe posibilități pentru stocarea variabilelor.

- stocarea variabilelor în memoria SRAM internă;
- stocarea variabilelor în setul de registre;
- stocarea variabilelor în memoria EEPROM internă (memorie nevolatilă).

Prima metodă se aplică de regulă variabilelor globale sau celor locale declarate în funcția main(). Tipul acestora decide dimensiunea alocată automat în memorie iar adresa este stabilită de compilator.

A doua metodă este aplicată variabilelor locale declarate în interiorul funcțiilor, parametrilor unei funcții sau valorilor returnate de o funcție. Datorită faptului că spațiul în setul de registre este foarte redus, în condițiile ocupării acestuia compilatorul va proceda la alocarea unor zone suplimentare în memoria RAM internă. De aceea este de dorit evitarea construirii de funcții cu mai mult de doi parametri sau folosirea excesivă a variabilelor locale. În schimb se poate folosi transmiterea parametrilor prin pointeri sau referință și variabile globale.

Datorită limitărilor care sunt prezente în memoria de date (4k la ATMEGA128) se recomandă încadrarea corectă a variabilelor în domeniul în care sunt utilizate, folosirea transmiterii parametrilor prin referință sau utilizarea unor variabile temporare declarate global. De asemenea, având în vedere faptul că stiva hardware este salvată tot în memoria SRAM internă, se va evita folosirea algoritmilor recursivi.

c. Stocarea constantelor

Dacă nu se specifică altfel, constantele se vor depune tot în memoria SRAM. Pentru optimizarea folosirii acesteia, se poate seta pentru depunerea constantelor în memoria de program punând declarația `__flash` în față.

Aplicație:

Pentru exemplificare vom continua cu pașii descriși în laboratorul anterior. Să se creeze un proiect la care se adaugă fișierul `ma_io.c` creat cu utilitarul `MakeApplication`. Să se creeze apoi un fișier nou, intitulat `main.c`.

Fișierele librărie generat cu `make_application` au nevoie de niște modificări pentru a putea fi utilizate în interiorul IAR. Astfel în interiorul fișierelor `.c` se vor face următoarele modificări:

- se va comenta (șterge) incluziunea `usercode.h` de la începutul fișierului;
- se vor comenta (șterge) următoarele macro-funcții care încep cu: `ENTER_MA_...`, `EXIT_MA_...` și `INSIDE_MA_...`

În fișierul `main.c` se vor scrie următoarele:

```
#include "ma_io.h" //--1

void main( )/--2
{
    MA_Init_IO( );/--3
    MA_WritePort_IO(MA_PORTA,0xff,0xa8);/--4
    for(;;);/--5
}
```

--1: are loc incluziunea fișierului `.h` al librăriei. În cazul în care fișierul `ma_io.h` nu se află în același director cu fișierul `main.c` trebuie introdusă calea completă.

--2: așa se declară începutul programului principal, execuția programului începe cu apelul acestuia.

--3: este apelată funcția care inițializează liniile portului la setările inițiale configurate cu `MakeApplication`.

--4: este apelată funcție care setează toate liniile de ieșire ale portului în 1.

--5: orice program trebuie să se termine într-o buclă infinită.

Se include fișierul în proiect, așa cum a fost prezentat mai sus și apoi se realizează compilarea.

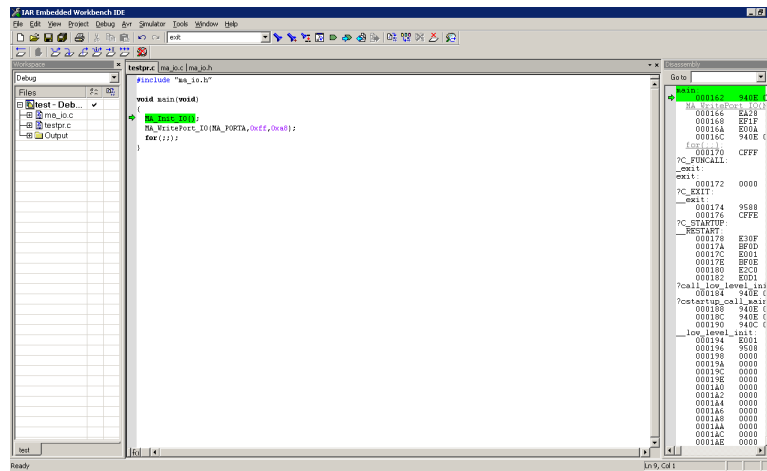
3. Simularea comportamentului microconroller-ului folosind IAR Embedded Workbench

După compilarea programului și corectarea erorilor urmează executarea și testarea acestuia. O metodă de testare ar fi simularea comportamentului. Pentru aceasta, se selectează instrumentul de testare ca fiind simulatorul și se deschide utilitarul de simulare C-Spy integrat în IAR prin selectarea din meniu Project – Debug sau apăsarea butonului



corespunzător din bara cu butoane:

La rularea utilitarului de depanare se va afișa fereastra de testare/depanare care arată ca în figura de mai jos.



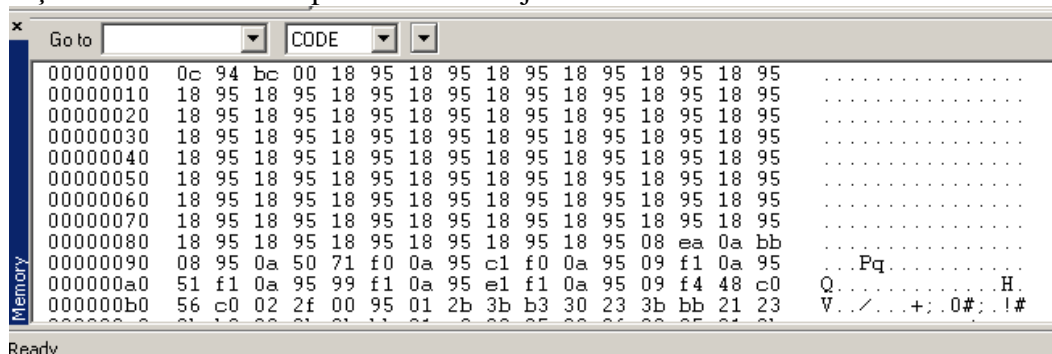
În partea stângă a ferestrei este afișată fereastra de dezasamblare în care se poate observa echivalentul în instrucțiuni asamblare ale comenzilor în C date.

La rularea aplicației cursorul (verde) este poziționat pe prima instrucțiune. Pentru rularea programului se pot acționa unul din butoanele tipice pentru mediile de testare și depanare software:

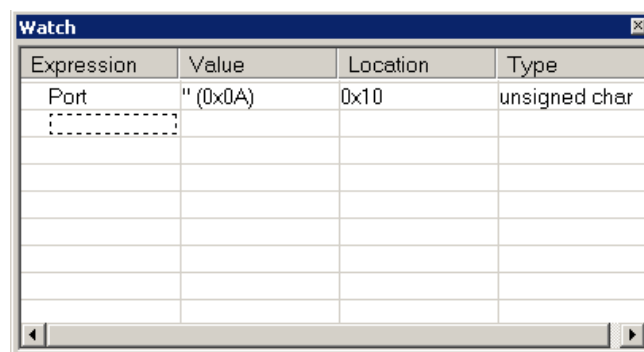
Buton	Explicație
	Reset : cursorul revine la prima instrucțiune
	Step over : se execută un pas echivalent cu o instrucțiune. În cazul în care este un apel de funcție nu va intra în funcție
	Step into : se execută un pas echivalent cu o instrucțiune. În cazul în care este un apel de funcție cursorul se va poziționa pe prima instrucțiune în funcție
	Step out : se execută un pas echivalent cu o instrucțiune. În cazul în care cursorul este poziționat într-o funcție iese din aceasta și se poziționează pe prima instrucțiune după

	apelul funcției
	Next statemet: execută următoarea instrucțiune
	Run to cursor: execută toate instrucțiunile până la cursor sau până întâlnește un break point
	Go : rulează programul până la întâlnirea unui break point. Cursorul se poziționează pe break point
	Breakpoint : introduce un punct care va suspenda execuția programului când va ajunge aici.
	Stop debugging : oprește testare, revine în mediul IAR
	Make and debug : compilează eventualele schimbări făcute la codul sursă și reia testarea

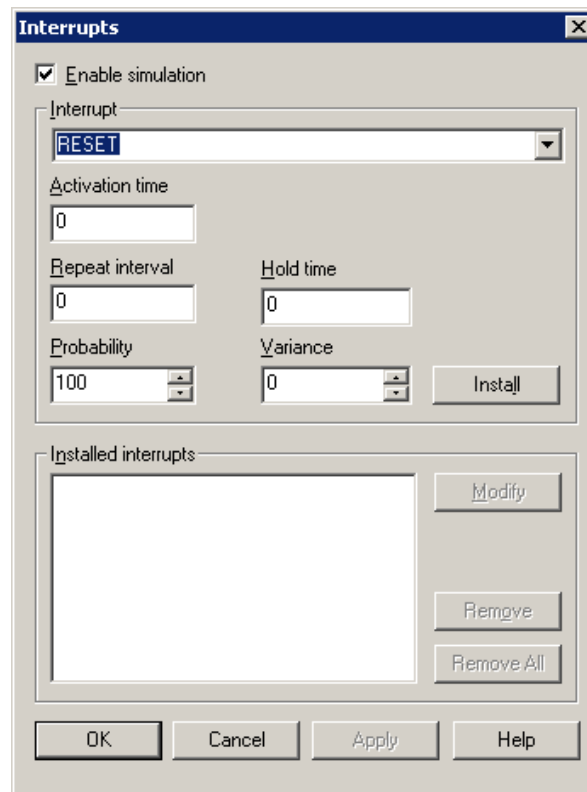
Pentru vizualizarea datelor rezultate din simulator se selectează din meniu View. Aici se pot selecta mai multe date de monitorizat. Pentru monitorizarea memoriei SRAM, a memoriei de program precum și a spațiului IO se selectează din meniu opțiunea Memory. Se afișează o fereastră de tipul celei de mai jos:



Pentru afișarea variabilelor precum și a adreselor alocate, se folosește View – Watch și se introduc aici variabilele care se doresc prin tragere din codul sursă.



Simularea comportamentului unei surse de întrerupere se face în felul următor. Se selectează din meniu Simulator – Interrupts și se accesează o fereastră de tipul celei de mai jos.



Din interrupt se selectează sursa de la care poate apare cererea de întrerupere;
Activation time: reprezintă timpul de activare în ciclui, după care va apărea cererea de întrerupere;
Repeat interval: intervalul de timp în care se va repeta apariția întreruperii
Hold time: timpul dat în ciclui, în care întreruperea rămâne activată dacă nu a fost tratată;
Probability: probabilitatea de apariție a întreruperii, specificată în procente;
Variance: variația în timp, ca procent din perioada de repetiție a întreruperii, în care întreruperea poate apare. De exemplu, dacă intervalul de repetiție este setat la 100 o variație de 5% poate semnifica faptul că întreruperea poate apărea aleatoriu între o perioadă de 95 și 105 pentru a simula o variație a timpului.
Deși simulatorul nu realizează o raportare la timpul fizic a programului care se execută el poate fi utilizat pentru testarea funcționalității modulelor.

Aplicație:

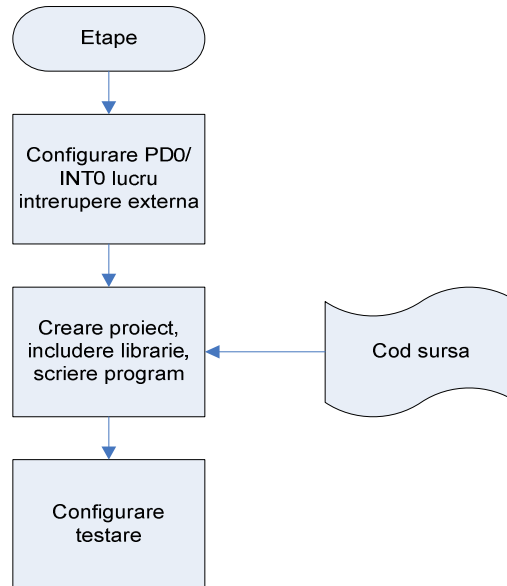
Pentru programul scris anterior să se realizeze simularea acestuia. Să se parcurgă programul afișându-se spațiul IO în care se va observa modificarea stării pinilor. Să se afișeze variabila – parametru Port din interiorul funcției MA_WritePort_IO și să se determine care este adresa care i-a fost alocată.

Probleme rezolvate:

1. Pe pinul PD0/INT0 al unui sistem cu AVRATMega128 este conectat un generator de semnal dreptunghiular. Să se realizeze un program care realizează numărarea acestor impulsuri.

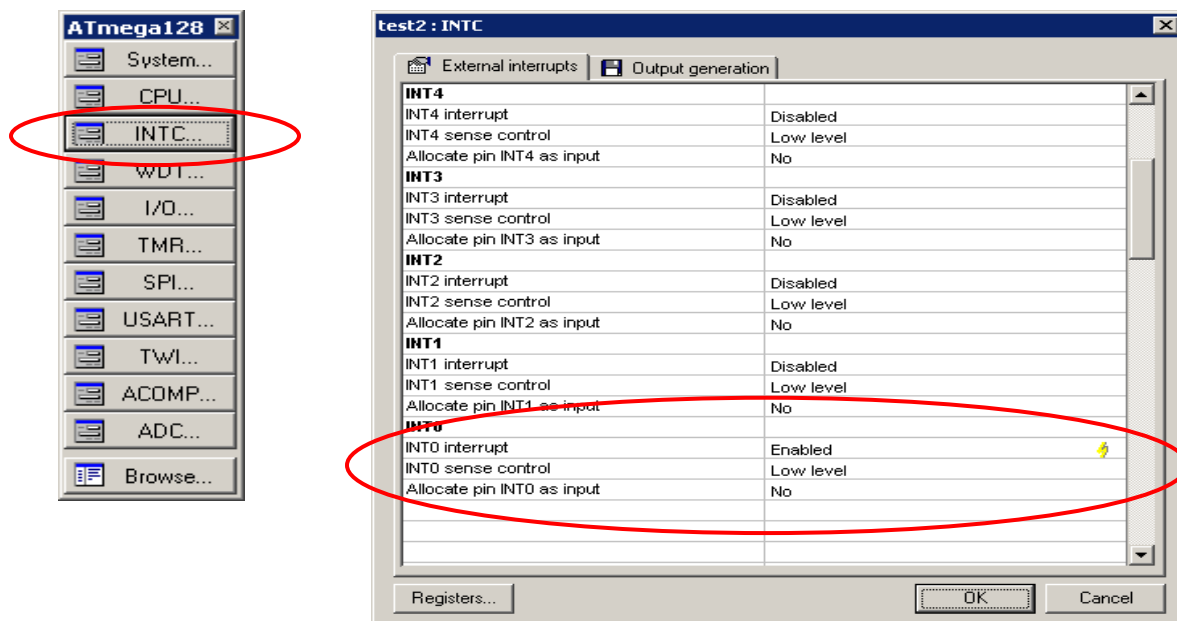
Rezolvare:

Se va lucra cu întreruperi generate extern pe pinul INT0. În subrutina de tratare a întreruperii se va plasa o variabilă care va contoriza aceste impulsuri.



Etape necesare:

1. Configurarea circuitului cu IAR MakeApp pentru generarea de cereri de întrerupere de la pinul INT0



În urma compilării sunt generate fișierele `ma_intc.c` și `ma_intc.h`.

2. Se construiește un proiect în IAR EW AVR, se include fișierul `ma_intc.c` în proiect. Se modifică fișierul `ma_intc.c` astfel încât acesta să poată fi compilat cu IAR. Se creează un fișier nou care se numește `main.c`. În acesta se va scrie următorul cod:

```
#include "ma_intc.h" //--1
#include <inavr.h> //--2

unsigned int contor=0; //--3
void main()
{
    MA_Init_INT0(); //--4

    __enable_interrupt(); //--5
    for(;;) //--6

}
```

- 1: inclus pentru folosirea intreruperii externe
- 2: inclus pentru apelarea functiei de activare a intreruperilor
- 3: variabilă globală în care se va contoriza, inițial are valoarea 0
- 4: initializare întreruperi externe
- 5: activare lucrul cu întreruperi
- 6: buclă principală.

În fișierul `ma_intc.c` se va introduce la început declarația variabilei:

```
extern unsigned int contor;
```

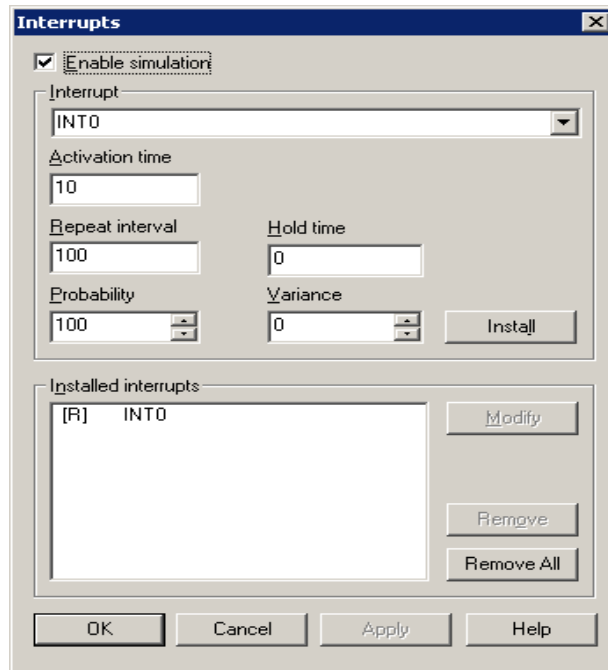
prin care se informează compilatorul că se utilizează o variabilă declarată în alt fișier din proiect (în cazul de față în `main`).

În interiorul subrutinei de tratare a întreruperii (funcția `MA_IntHandler_INT0_INT0`) se va introduce:

```
contor++;
```

prin care se incrementează valoarea contorului de impulsuri.

3. Se va compila programul și se va rula simulatorul. Se va selecta simularea generatorului de întreruperi pentru întreruperea externă 0, cu o perioadă de 100 cicluri așa cum se observă în figura de mai jos:



Se va monitoriza valoarea variabilei contor și se va observa poziția acesteia în interiorul memoriei de date. Se va introduce un break point în subrutina de tratare a întreruperii la contorizarea variabilei.

Care pin paralel general IO nu se va mai putea utiliza în cazul folosirii pe post de pin întrerupere externă?

Probleme de rezolvat:

1. Să se realizeze un program care să permită contorizarea separată a trei semnale dreptunghiulare. Se vor folosi pinii de întrerupere INT0, INT1 și INT2. Se vor prezenta variabilele contor monitorizate în paralel. Semnalele externe vor fi generate la frecvențe diferite în simulator.
2. Pe pinul PD1/INT1 al unui microcontroler AVRATMega128 se aplică un semnal dreptunghiular periodic cu frecvența f_x . Să se realizeze un program care să transmită pe pinul PB0 un semnal cu frecvența $f_x/4$ și factor de umplere $1/4$.